

**Multi – level linked list**  
**–Delete At Position(Main List)**  
**–Space Complexity And Algorithm**

**Program: –**

```
struct Node
{
    int data;
    struct Node *next; // Points to the right (Same Level)
    struct Node *child; // Points down (Sub Level)
};

Node *head = nullptr ; // Global Head Pointer
// Helper: Create a new node with malloc
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}

void deleteAtPosition()
{
    int pos;
    cout << "Enter position to delete: ";
    cin >> pos;

    if (head == nullptr)
        return;

    Node *temp = head;
```

```

    if (pos == 1)
    {
        head = head->next;
        free(temp);
        return;
    }

    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr || temp->next == nullptr)
    {
        cout << "Position out of range." << endl;
        return;
    }

    Node *toDelete = temp->next;
    temp->next = toDelete->next;
    free(toDelete);
    cout << "Deleted at position " << pos << endl;
}
...
int main()
{
    int choice;
    while (true)
    {
        cout << "7. Delete at Position (Main)" << endl;
        cin >> choice;

        switch (choice)
        {
            case 7:
                deleteAtPosition();
                break;

```

```
        default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

## *Space Complexity Analysis*

### *Input Space Complexity*

#### *A. Parameter – only Input Space Complexity*

*This method considers ONLY:*

- *Function parameters*
- *Explicit external inputs*
- *Primitive input values*

*It does NOT include the entire linked list structure.*

##### *Function Parameters*

```
void deleteAtPosition()
```

*This function has: No formal parameters.*

##### *User Input*

```
int pos;
cin >> pos;
```

*Only one integer input is taken.*

*Memory for one integer:*

$$\theta(1)$$

***Final Parameter – only Input Space Complexity***

*Only one integer (pos) is externally supplied.*

*Therefore:*

$$\boxed{\theta(1)}$$

## ***B. Full Data Input / Holistic Input Space Complexity***

*This approach considers:*

*Entire actual data structure stored in memory and operated on by the function.*

*Here the linked list itself is the input data.*

### ***Linked List Structure***

```
struct Node
{
    int data;
    struct Node *next; // Points to the right (Same Level)
    struct Node *child; // Points down (Sub Level)
};
```

*Each node contains:*

- *one integer*
- *two pointers*

*Each node occupies constant memory.*

*So:*

***One Node =  $\theta(1)$***

## Total Nodes

Suppose the multilevel linked list contains:

$n$  nodes

Then total input memory:

$$\begin{aligned} n \times \Theta(1) \\ = \Theta(n) \end{aligned}$$

## Why Child Nodes Are Counted

Even though `deleteAtPosition()` traverses only:

```
temp = temp->next;
```

the entire multilevel linked list already exists in memory.

Each node includes:

```
Node *child; // Points down (Sub Level)
```

Thus child – linked nodes are also part of the stored input structure.

Hence total stored data:

$$\Theta(n)$$

where:

$n$  = total nodes across all levels.

## Final Holistic Input Space Complexity

$$\boxed{\Theta(n)}$$

## 2. Auxiliary Space Complexity

*Auxiliary Space means:*

*Extra temporary memory used during execution excluding the input data itself.*

### *Variables Used*

*Inside the function:*

```
int pos;  
Node *temp;  
Node *toDelete;
```

*Also loop variable:*

```
int i = 1;
```

### *Memory Used by Variables*

*Each variable occupies constant memory.*

*Thus:*

$$\theta(1)$$

### *Does the Loop Increase Space?*

```
for (int i = 1; i < pos - 1 && temp != nullptr; i++)  
{  
    temp = temp->next;  
}
```

*The loop traverses many nodes,  
BUT traversal does not allocate additional memory.*

*Only one pointer (temp) moves.*

*Hence space does NOT grow with n.*

# ***Dynamic Memory Allocation?***

*Inside function:*

```
free(toDelete);
```

*Memory is deallocated, not allocated.*

*No new dynamic memory is created.*

*Thus no extra growing memory usage.*

## ***Recursion?***

*No recursion exists.*

*Therefore:*

- *no recursive stack*
- *no additional call frames proportional to  $n$*

## ***Final Auxiliary Space Complexity***

$$\boxed{\theta(1)}$$

---

# Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(1)}_{\text{Input}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(n)}_{\text{Input}} = O(n).$$

**1) If Asked “Total Space Complexity” then natural answer is :  $O(n)$  [*Full Data + Auxiliary*]**

**2) If not asked total space complexity then:**

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ( $O(1)$ )** is the expected answer.



In most academic DSA contexts and coding interviews, when someone simply asks:

“What is the space complexity?”

they usually mean:

|                            |
|----------------------------|
| Auxiliary Space Complexity |
|----------------------------|

unless they explicitly say:

- total space complexity
- overall memory usage
- data structure space
- input space
- storage complexity

So for your insertion function, the expected answer is usually:

|        |
|--------|
| $O(1)$ |
|--------|

because only constant extra working memory is used.

---

**Algorithm:**

***MultiDelAtPos():***

***1. Declare integer pos.***

***2. Write: ``Enter position to delete: ``***

***3. Read pos.***

***4. [if list is Empty]***

***if START = NULL, then:***

***return void and exit.***

***[End of If]***

***5. [if List is not Empty]***

***SET TEMPPTR := START***

***6. [Delete Node at 1st position]***

***if pos = 1, then:***

***SET START := LINK[START] This will make HEAD to  
point at NEXT Node.***

***Call FREE(TEMP)[FREE NODE AT 1st Position]***

***Return void and exit.***

***[END OF IF ]***

***7. if position is found then Traverse the list to the  
previous node of the node  
that is to get deleted***

***Repeat for i = 1 to pos – 2 AND TEMPPTR ≠ NULL:***

***SET TEMPPTR := LINK[TEMPPTR]***

8. *[If position is not found while traversal]*

*if TEMP\_PTR = NULL OR LINK[TEMP\_PTR] = NULL\_PTR, then:*

*Write : ``Position is out of range.``*

*Return void and exit.*

*[END of If Structure]*

9. *SET TODELETE\_PTR := LINK[TEMP\_PTR]*  $\left[ \begin{array}{l} \text{LINK TODELETE} \\ \text{Pointer to the node} \\ \text{i. e. to get deleted.} \end{array} \right]$

10. *SET LINK[TEMP\_PTR] := LINK[TODELETE\_PTR]*  $\left[ \begin{array}{l} \text{This will link} \\ \text{PREVIOUS} \\ \text{NODE of the} \\ \text{Deleted Node} \\ \text{to the next} \\ \text{node of} \\ \text{Deleted Node.} \end{array} \right]$

11. *CALL FREE(TODELETE\_PTR)*  $\left[ \begin{array}{l} \text{Free the allocated memory} \\ \text{pointed by TODELETE} \\ \text{POINTER.} \end{array} \right]$

12. *Write: ``Deleted at position`` , pos .*

13. *[END OF ALGORITHM]*

---